

Frankfurt University
of Applied Sciences

– Fachbereich 2: Informatik und Ingenieurwissenschaften –

Deep-Learning-basierte Anomalieerkennung in Passantenfrequenzen

Projektarbeit im Rahmen des Bachelorstudiums

Bachelor of Science (B.Sc.)

vorgelegt am 10. August 2026 von

Sherwin Nourinfar, Wassim Hanane

Matrikelnummer: 1552887, 1583937

Referent	:	Daniel Helmer
Korreferent	:	–

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Frankfurt, 10. August 2026

Your Signature

Sherwin Nourinfar, Wassim Hanane

Inhaltsverzeichnis

1 Einführung **2**

1.1 Motivation . . . 2

1.2 Zielsetzung . . . 2

1.3 Aufbau der Arbeit . . . 2

2 Hintergrund **3**

2.1 Zeitreihen und Anomalien . . . 3

2.2 Erkennung mit Vorhersagen . . . 3

2.3 LSTM, Autoencoder und Transformer . . . 3

3 Stand der Technik **4**

3.1 Prognosebasierte Anomalieerkennung . . . 4

3.2 Transformer für Zeitreihenprognosen . . . 4

3.3 Anomalieerkennung durch Rekonstruktion . . . 4

3.4 Fairer Vergleich . . . 5

4 Entwickelte Methode **6**

4.1 Datengrundlage . . . 6

4.2 Datenqualität . . . 6

4.3 Bereinigung der Mehrfachmessungen . . . 7

4.4 Behandlung der Messlücken . . . 8

4.5 Kritische Betrachtung . . . 9

4.6 Forecaster . . . 10

4.6.1 Lernaufgabe und Eingaberepräsentation . . . 10

4.6.2 Architektur und Größenwahl . . . 11

4.6.3 Nichtkonformitätsscore und Intervallkalibrierung . . . 12

4.6.4 Kausale Alarmregel . . . 12

4.6.5 Umgang mit imputierten Zielwerten . . . 12

4.7 Transformer . . . 12

4.8 Autoencoder . . . 13

4.9 Baseline . . . 14

5 Evaluation der Methode **15**

5.1 Zeitliche Datentrennung 15

5.2 Bewertung der Prognosegüte 16

5.3 Anomalieinjektion 16

5.4 Punkt- und Ereignismetriken 17

5.5 Reproduzierbarkeit 17

6 Ergebnisse **19**

6.1 Training und Detektorkalibrierung 19

6.2 Prognosegüte im Testabschnitt 20

6.3 Kandidaten im unveränderten Datensatz 21

6.4 Ergebnisse der Anomalieinjektion 21

6.5 Vergleich der finalen Modelle 22

6.6 Diskussion und Limitationen 23

7 Zusammenfassung und Ausblick **24**

Literatur **25**

Kurzfassung

Diese Arbeit entwickelt ein unüberwachtes Verfahren zur Erkennung auffälliger Passantenfrequenzen aus Bad Nauheim. Die Verweildauer bleibt eine zusätzliche Modellgröße, löst jedoch keinen Alarm aus. Der untersuchte API-Snapshot umfasst 30 Sensoren vom 30. Juni 2025 bis zum 30. Juli 2026. Die Datenaufbereitung verdichtet 572.461 Rohdatensätze zu 284.745 Sensorstunden, behandelt 18 gemeinsame Nullausfälle als fehlend und ergänzt 98.137 Stunden ausschließlich aus Vergangenheit oder Trainingsprofilen. Imputierte Werte bleiben als gekennzeichneter Kontext erhalten, werden aber nicht als Lern- oder Bewertungsziel verwendet.

Untersucht werden ein Quantil-LSTM mit 21.374 Parametern, ein Quantil-Transformer mit 61.062 Parametern, ein LSTM-Autoencoder mit 50.562 Parametern und ein parameterfreier robuster $t-24$-Detektor. Eine breite, chronologische Modellsuche wird per Successive Halving verkürzt; die beiden Finalisten jedes neuronalen Verfahrens laufen bis zum vollständigen Early-Stopping-Vergleich. Bei starken künstlichen Anomalien erreicht der LSTM-Forecaster einen mittleren Punkt-F1 von 0,653 und einen Ereignis-F1 von 0,885. Der Transformer erreicht 0,603 beziehungsweise 0,824, der Autoencoder 0,459 beziehungsweise 0,547 und die saisonale Methode 0,326 beziehungsweise 0,478. Der Transformer liefert die genaueren Holdout-Prognosen und die knapp höchste mittlere ROC-AUC, während die kalibrierte LSTM-Alarmregel den höchsten Ereignis-F1 erzielt. Im unveränderten Testabschnitt markieren LSTM und Transformer 105 beziehungsweise 91 Stunden in 79 beziehungsweise 71 Ereignissen; 11.636 fehlende Messungen werden in 1.460 Datenqualitätsereignissen getrennt ausgewiesen. Die synthetischen Tests belegen die technische Erkennungsfähigkeit für definierte Veränderungen. Fehlende vollständige reale Markierungen und eine räumlich unabhängige externe Datenquelle begrenzen weiterhin Aussagen über unbekannte reale Anomalien.

Kapitel 1

Einführung

Wie viele Menschen zu einer bestimmten Zeit an einem Ort unterwegs sind, folgt meist einem klaren Rhythmus. Tagsüber sind es mehr als nachts, und unter der Woche andere Zahlen als am Wochenende. Auch der Ort spielt eine Rolle, denn an einem belebten Platz ist mehr los als in einer ruhigen Seitenstraße. Wenn man dieses regelmäßige Verhalten kennt, fallen Abweichungen davon auf. Eine solche Abweichung vom gewohnten Muster nennt man Anomalie.

Grundlage dieser Arbeit ist ein realer Datensatz der Stadt Bad Nauheim. An 30 Standorten wurde vom 30. Juni 2025 bis zum 30. Juli 2026 stündlich gezählt, wie viele Passanten unterwegs waren. Für jeden Sensor entsteht so eine eigene Zeitreihe über rund 13 Monate.

1.1 Motivation

Mit diesem Projekt wollen wir herausfinden, ob sich solche Anomalien mit Methoden des Deep Learnings zuverlässig erkennen lassen und welche Modellart sich dafür am besten eignet. Die Schwierigkeit dabei ist, dass die Daten nicht beschriftet sind. Es ist nirgends vermerkt, welche Stunde normal war und welche nicht. Ein Modell muss das normale Verhalten also selbst aus den Daten lernen und melden, wenn die Realität davon abweicht. Feste Schwellenwerte helfen hier kaum, weil sich das Normalverhalten je nach Tageszeit, Wochentag und Standort verändert.

1.2 Zielsetzung

Das Ziel ist ein System, das für jeden Sensor das normale Passantenaufkommen lernt und auffällige Stunden meldet. Dafür werden ein LSTM-Forecaster, ein Transformer-Forecaster und ein LSTM-Autoencoder umgesetzt und anhand derselben künstlichen Anomalien und Kennzahlen verglichen. Eine robuste saisonale Methode bildet die einfache Baseline. So lässt sich trotz fehlender realer Anomalielabels begründet prüfen, welcher der tatsächlich getesteten Ansätze für diese Daten besser geeignet ist.

1.3 Aufbau der Arbeit

Kapitel 2 erklärt die nötigen Grundlagen zu Zeitreihen, neuronalen Netzen und den verwendeten Architekturen. Anschließend ordnet der Stand der Technik die Arbeit in bestehende Verfahren ein. Danach folgen die entwickelte Methode mit Datenaufbereitung und Training, die Evaluation der Modelle und die Ergebnisse. Den Abschluss bildet eine Zusammenfassung mit Ausblick.

Kapitel 2

Hintergrund

2.1 Zeitreihen und Anomalien

Eine Zeitreihe ordnet Messwerte chronologisch. Bei Passantenfrequenzen hängen benachbarte Stunden zusammen; zusätzlich wiederholen sich Tagesmuster. Eine Anomalie ist deshalb nicht zwingend ein großer absoluter Wert, sondern eine Abweichung vom zeit- und ortsabhängig erwarteten Verhalten. Punktanomalien betreffen einzelne Stunden. Sprunghafte Niveauwechsel (Level-Shifts), schleichende Verschiebungen (Drifts) und festgefrorene Sensorwerte (Stuck-at) erstrecken sich dagegen über mehrere Stunden.

Im vorliegenden Datensatz fehlen vollständige reale Anomaliekennzeichnungen. Das Lernen des Normalverhaltens erfolgt daher selbstüberwacht: Aus vorhandenen Messwerten wird ein künstliches Lernziel erzeugt, ohne Klassen wie "normal" oder "anomal" vorzugeben. Erst der Fehler zwischen Modellschätzung und Messung wird als Anomaliesignal verwendet.

2.2 Erkennung mit Vorhersagen

Ein Vorhersagemodell erhält die letzten w Zeitpunkte und schätzt den nächsten Wert $\hat{x}_t$. Ein großer Prognosefehler $|x_t - \hat{x}_t|$ deutet darauf hin, dass die Beobachtung nicht zum erlernten Verlauf passt. Zusätzliche Messgrößen können beim Lernen als Kontext dienen, ohne selbst Teil der Alarmentscheidung zu sein. Vorhersagegüte und Erkennungsleistung sind dabei nicht gleichbedeutend: Ein kleiner mittlerer Fehler garantiert noch keine klare Trennung seltener Anomalien.

2.3 LSTM, Autoencoder und Transformer

Long Short-Term Memory (LSTM) ist eine rekurrente Netzstruktur mit steuerbaren Speicherzuständen und eignet sich zur Verarbeitung geordneter Sequenzen [4]. Ein LSTM kann ein Zeitfenster zu einem Zustandsvektor verdichten und daraus eine Prognose erzeugen.

Ein Autoencoder lernt dagegen, sein Eingabefenster zu rekonstruieren. Ein hoher Rekonstruktionsfehler kann eine unbekannte Struktur anzeigen. Prognose und Rekonstruktion beruhen somit auf unterschiedlichen Annahmen über normales Verhalten. Ein Transformer verarbeitet verschiedene Zeitpunkte mit Self-Attention und kann dadurch auch weiter auseinanderliegende Zusammenhänge direkt gewichten. Da der Mechanismus allein keine Reihenfolge kennt, wird die Position jedes Zeitpunkts zusätzlich kodiert [21]. Diese zusätzliche Freiheit ist vor allem bei längeren Reihen und einer größeren Datenbasis nützlich.

Kapitel 3

Stand der Technik

3.1 Prognosebasierte Anomalieerkennung

Hundman et al. verwenden LSTM-Prognosefehler und dynamische Schwellen zur Erkennung von Anomalien in Zeitreihen [5]. Globale rekurrente Prognosemodelle können Informationen über mehrere verwandte Zeitreihen teilen und zugleich reihenspezifische Merkmale berücksichtigen [14]. Quantilprognosen bilden zusätzlich die Unsicherheit einer Vorhersage ab, statt nur einen einzelnen Schätzwert auszugeben [3, 8]. Dropout kann die Anpassung eines Netzes an seine Trainingsbeispiele begrenzen [18].

Diese Arbeiten begründen die im Projekt verwendeten Bausteine Prognosefehler, gemeinsames Lernen, Sensorinformation, Quantilausgabe und Dropout. Sie geben jedoch keine optimale Schichtbreite für den kurzen Datensatz aus Bad Nauheim vor. Die konkreten Größen werden deshalb in Abschnitt 4.6.2 anhand von Datenumfang und Parameterbudget festgelegt.

3.2 Transformer für Zeitreihenprognosen

Der ursprüngliche Transformer verbindet Multi-Head-Self-Attention mit Residualpfaden, positionsweisen Feedforward-Netzen und einer sinusförmigen Positionskodierung [21]. Xiong et al. zeigen, dass eine Layer-Normalisierung vor Attention und Feedforward-Netz stabilere Gradienten ermöglicht; diese Pre-LN-Anordnung reduziert zugleich die Notwendigkeit einer gesonderten Lernraten-Aufwärmphase [23].

Für lange Prognosefenster fasst PatchTST benachbarte Zeitpunkte zu Tokens zusammen und verkleinert dadurch die Attention-Matrix [12]. Das vorliegende Fenster umfasst jedoch nur 24 Stunden. Damit einzelne Spikes und Dips erhalten bleiben, wird deshalb jede Stunde als eigenes Token verwendet. Die guten Ergebnisse einfacher linearer Zeitreihenmodelle von Zeng et al. motivieren zusätzlich einen direkten linearen Pfad von den historischen Messwerten zur Prognose [24]. Er ergänzt die Attention, ersetzt sie aber nicht. Statische Sensorinformation, bekannte Kalendergrößen und Quantilausgaben orientieren sich an den Eingabearten probabilistischer Zeitreihen-Transformer [9]. Die konkrete Breite bleibt wegen des kleinen Datensatzes deutlich unter den Architekturen dieser Arbeiten und wird auf einem getrennten Tuningabschnitt gewählt.

3.3 Anomalieerkennung durch Rekonstruktion

Ein Autoencoder lernt, normale Eingabefolgen zu komprimieren und anschließend wiederherzustellen. Große Rekonstruktionsfehler dienen danach als Hinweis auf eine Abweichung. Malhotra et al. übertragen dieses Prinzip auf mehrdimensionale Sensordaten. Ihr Modell verwendet jeweils eine LSTM-Schicht im Encoder

und im Decoder und zeigt, dass ein solcher Aufbau sowohl periodische als auch weniger gut vorhersagbare Reihen verarbeiten kann [11]. Die Arbeit begründet den verwendeten Grundaufbau, gibt aber keine allgemein optimale Zahl von LSTM-Einheiten für andere Datensätze vor.

Denoising-Autoencoder erhalten beim Training eine teilweise verdeckte Eingabe und sollen trotzdem die unveränderten Werte rekonstruieren. Dadurch wird das reine Kopieren der Eingabe erschwert [22]. Neuere Verfahren verwenden ebenfalls zeitliche Maskierung, kombinieren diese jedoch teilweise mit mehreren Transformern, Frequenzdarstellungen und zusätzlichen Fehlerfunktionen [2]. Für den kontrollierten Vergleich und das kurze 24-Stunden-Eingabefenster dieses Projekts wird nur das einfache Maskierungsprinzip übernommen.

3.4 Komplexität und fairer Vergleich

Aktuelle Vergleichsarbeiten warnen davor, eine komplexere Architektur automatisch als besseren Anomaliedetektor zu betrachten. Sarfraz et al. zeigen, dass zusätzliche Komplexität häufig nur geringe Vorteile bringt, und empfehlen einfache, verständliche Vergleichsmodelle [15]. Der umfangreiche TSB-AD-Vergleich kommt ebenfalls zu dem Ergebnis, dass einfache neuronale und statistische Verfahren viele moderne Netze übertreffen können [10].

Deshalb wird kein Anspruch erhoben, mit einer bestimmten Schichtgröße einen allgemein besten Stand der Technik zu erreichen. Stattdessen werden Literaturbausteine auf kompakte LSTM-, Transformer- und Autoencoder-Modelle übertragen und mit denselben Zeitabschnitten, künstlichen Anomalien und Kennzahlen geprüft. Ein robuster saisonaler Detektor ohne trainierbare Parameter bildet zusätzlich die einfache Referenz. Eine zusätzliche Schicht wird nur dann gerechtfertigt, wenn sie für die jeweilige Lernaufgabe einen nachvollziehbaren Zweck erfüllt.

Kapitel 4

Entwickelte Methode

Dieses Kapitel beschreibt zunächst die reproduzierbare Aufbereitung der Rohdaten und anschließend die vier untersuchten Anomaliedetektoren. Methodische Entscheidungen werden von den später berichteten Ergebnissen getrennt dargestellt.

4.1 Datengrundlage

Die Rohdaten liegen in Form einer JSON-Datei mit 572.461 Einzelmessungen vor. Jede Messung gehört zu einem von 30 Sensorstandorten in der Innenstadt und im Kurpark von Bad Nauheim und enthält neben dem Zeitstempel die Anzahl der erfassten Passanten (`visitors`) sowie deren mittlere Verweildauer (`avgDuration`). Der globale Erfassungszeitraum reicht vom 30. Juni 2025, 00:00 Uhr, bis zum 30. Juli 2026, 11:00 Uhr UTC und umfasst 9.492 Stunden. Da nicht alle Sensoren während des gesamten Zeitraums Messungen lieferten, gilt ein Sensor ab seiner ersten Messung bis zum globalen Datenende als aktiv. Dadurch entstehen 284.745 aktive Sensorstunden. Ein ausbleibender Datenstrom am Reihenende bleibt damit als möglicher Sensorausfall sichtbar.

4.2 Datenqualität

Die explorative Analyse mit `analyse/analyse.py` untersucht zunächst Aufbau, Mehrfachmessungen, Verteilungen und Beziehungen der Messgrößen. Die anschließende Bildung eines vollständigen Stundenrasters in `analyse/preprocessing_plots.py` macht zusätzlich fehlende Stunden sichtbar. In vorhandenen Rohzeilen fehlen weder `visitors` noch `avgDuration`; eine ausgefallene Stunde fehlt jedoch als ganze Zeile und wäre mit einer reinen Suche nach leeren Einzelwerten nicht erkennbar.

Mehrfachmessungen: Für 147.072 Kombinationen aus Sensor und Stunde enthält der Datensatz mehrere Einträge, in der Spitze 34 Messungen für dieselbe Stunde (Abbildung 4.1). Die Werte dieser Einträge weichen teilweise voneinander ab, es handelt sich also nicht um exakte Kopien. Ohne Bereinigung gingen diese Stunden mehrfach und mit widersprüchlichen Werten in das Training ein.

Messlücken: Nach der Zusammenfassung der Mehrfachmessungen liegen zunächst 187.028 eindeutige Sensorstunden vor. In 18 Stunden melden mindestens zehn aktive Sensoren gleichzeitig exakt null Passanten und null Verweildauer. Diese gemeinsamen Nullmeldungen mit insgesamt 420 Sensorwerten werden als Systemausfälle behandelt. Damit verbleiben 186.608 gemessene Sensorstunden. Innerhalb der 284.745 aktiven Sensorstunden fehlen 98.137 Werte, entsprechend 34,46 %. Sie bilden 16.805 zusammenhängende Lücken. Davon umfassen 11.023 höchstens zwei Stunden; die längste Lücke dauert wegen eines ausbleibenden Sensors 1.248 Stunden (Abbildung 4.2). Zusätzlich zu den erkannten Nullausfällen existieren 1.791 Stunden, in denen kein zu diesem Zeitpunkt aktiver Sensor einen Rohwert liefert. Dies deutet auf gemeinsame Ausfälle

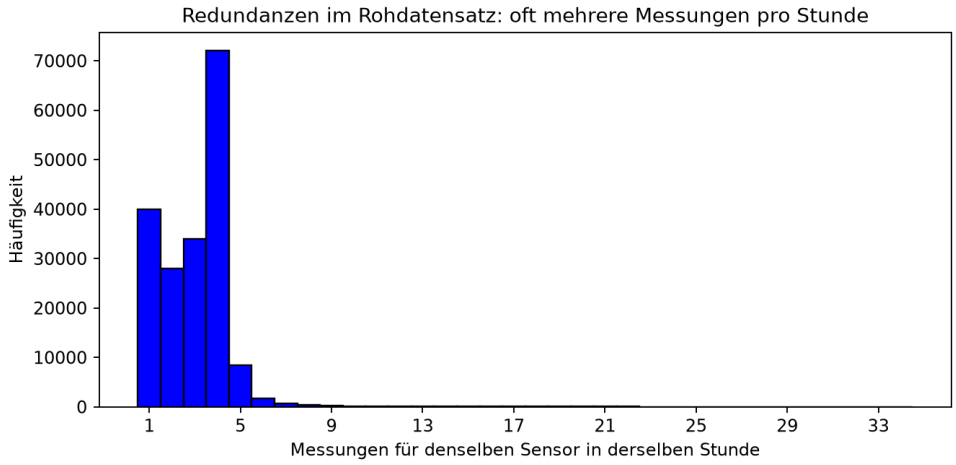


Abbildung 4.1: Häufigkeit von Mehrfachmessungen im Rohdatensatz. Für viele Stunden liegen mehrere, vereinzelt bis zu 34 Einträge desselben Sensors vor.

der Übertragungsinfrastruktur hin. Trotz dieser Lücken besitzen alle 30 Sensoren mindestens 72 gemessene Trainingsziele und können in die globalen Modelle eingehen.

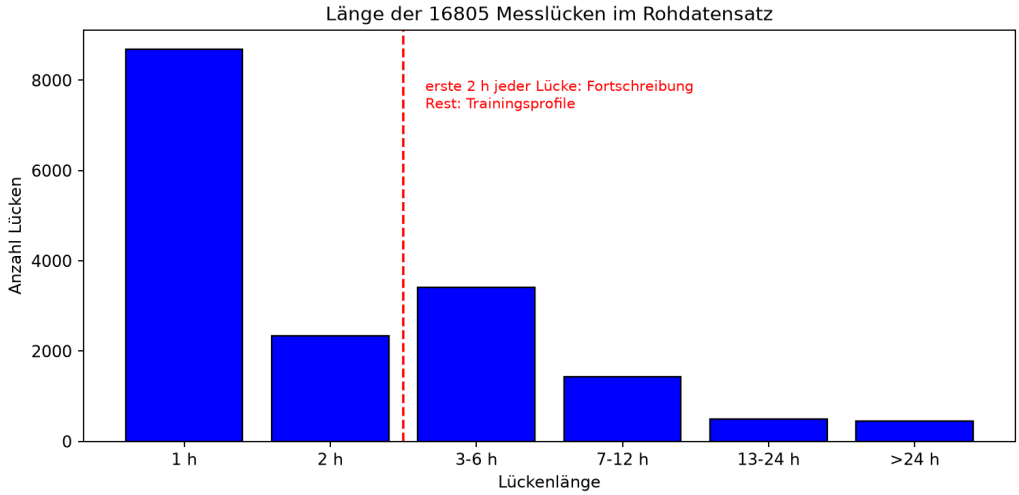


Abbildung 4.2: Längenverteilung der 16.805 Messlücken ab dem ersten Messwert eines Sensors. 11.023 Lücken umfassen höchstens zwei Stunden; Endausfälle bleiben in einer eigenen Klasse sichtbar.

Tabelle 4.1 fasst die wichtigsten Folgerungen der Analyse zusammen. Damit wird jede Aufbereitungsentscheidung aus einem beobachteten Datenmerkmal abgeleitet.

4.3 Bereinigung der Mehrfachmessungen

Alle Einträge derselben Kombination aus Sensor und Stunde werden zu einem Messpunkt zusammengefasst. Für die Passantenzahl und die Verweildauer wird jeweils der arithmetische Mittelwert gebildet, da keine der mehrfachen Messungen als verlässlichere Quelle gekennzeichnet ist. Dadurch entstehen 187.028 eindeutige Sensorstunden. Die 420 Werte der 18 gemeinsamen Nullmeldungen werden anschließend als fehlend markiert, sodass 186.608 Stunden als gemessen gelten. Allgemein greift diese Regel, wenn mindestens zehn Sensoren Werte liefern und sämtliche vorhandenen Werte gleichzeitig null Passanten und null Verweildauer melden. Nach Abschluss der Imputation wird die Passantenzahl auf ganze Personen und die Verweildauer

Tabelle 4.1: Aus der Datenanalyse abgeleitete Aufbereitung. Eigene Darstellung.

Analysebefund	Folgerung für die Aufbereitung
Bis zu 34 widersprüchliche Einträge je Sensorstunde	Zusammenfassung zu genau einem Messpunkt je Stunde
98.137 vollständig fehlende Stunden ab dem jeweiligen Messbeginn	Stundenraster, Imputation und dauerhafte Kennzeichnung der Ersatzwerte
18 gemeinsame Nullmeldungen von jeweils mindestens zehn aktiven Sensoren	Als gemeinsamen Systemausfall statt als echtes Passantenminimum behandeln
11.023 von 16.805 Lücken sind höchstens zwei Stunden lang	vergangenheitsbasierte Fortschreibung auf zwei Stunden begrenzen
Tages- und Wochenrhythmus sowie stark unterschiedliche Sensorniveaus	Kalendermerkmale, Zeitprofile und sensorweise Skalierung verwenden
Mittlere sensorweise Korrelation von nur 0,09 zwischen den Zielgrößen	`avgDuration` als zusätzliche Modellgröße beibehalten, den Alarm aber auf `visitors` ausrichten

auf zwei Dezimalstellen gerundet.

4.4 Behandlung der Messlücken

Die für das Training verwendeten Zeitfenster setzen zusammenhängende Stundenreihen voraus. Eine Lücke innerhalb eines Fensters würde andernfalls unbemerkt Werte aus weit auseinanderliegenden Zeiträumen kombinieren. Die Lücken werden deshalb vor dem Training aufgefüllt, und zwar für jeden Sensor getrennt, damit jeder Standort sein individuelles Niveau und Tagesprofil behält. Die zentrale Pipeline verwendet keine lineare Interpolation. Sämtliche Profile werden ausschließlich aus ursprünglich gemessenen Werten vor dem 1. Mai 2026, 00:00 Uhr UTC, geschätzt; Tuning-, Kalibrierungs- und Testwerte gehen damit nicht in die Profilbildung ein. Wochentag und Stunde werden in der lokalen Zeitzone `Europe/Berlin` bestimmt. Das Verfahren arbeitet in vier aufeinanderfolgenden Stufen.

Stufe 1, vergangenheitsbasierte Fortschreibung: Da 11.023 der 16.805 Lücken und damit 65,6 % höchstens zwei Stunden dauern, wird die Fortschreibung auf genau diesen Zeitraum begrenzt. Für die ersten zwei Stunden jeder Lücke wird der letzte zuvor gemessene Wert desselben Sensors übernommen. Mit t^- als Zeitpunkt der letzten Messung gilt

$$\hat{x}_t = x_{t^-}, \qquad 0 < t - t^- \leq 2 \text{ Stunden.} \tag{4.1}$$

Diese Regel gilt auch für die ersten beiden Positionen einer längeren Lücke. Sie benötigt keinen nachfolgenden Messwert und verhindert dadurch, dass ein späteres Prognoseziel in einen früheren Modellkontext zurückgespiegelt wird. Der Preis ist eine kurzfristig konstante Fortsetzung; die Regel stellt daher einen transparenten Kompromiss und kein allgemein optimales Imputationsverfahren dar. Auf diese Weise werden 24.929 Sensorstunden ergänzt.

Stufe 2, Wochentag-Stunden-Profil: Für die verbleibenden Lücken wird der Mittelwert desselben Sensors am gleichen lokalen Wochentag und zur gleichen lokalen Stunde aus gemessenen Trainingswerten eingesetzt. Historische Zeitprofile erhalten tages- und wochenzeitabhängige Strukturen und werden auch für fehlende Verkehrszähldaten untersucht [19]. Diese Stufe füllt 73.208 Sensorstunden.

Stufe 3, Stundenprofil: Existiert für eine Kombination aus Wochentag und Stunde keine Trainingsreferenz, wird der Mittelwert des Sensors zur gleichen lokalen Uhrzeit über alle Trainingstage verwendet. Im Jahresdatensatz ist diese Rückfallstufe nicht erforderlich.

Stufe 4, Sensormittel: Fehlt auch für die betreffende Uhrzeit eine Referenz, wird der Mittelwert sämtlicher gemessener Trainingswerte dieses Sensors eingesetzt. Auch diese Rückfallstufe ist nicht erforderlich. Da-

mit werden alle 98.137 Lücken ab der ersten Messung geschlossen; Zeiten vor der ersten Messung bleiben außerhalb des Datensatzes. Ergänzte Werte nach der letzten Messung bleiben ausdrücklich als Datenqualitätsproblem markiert.

Abbildung 4.3 kennzeichnet für zwei Sensoren die ergänzten Stunden und zeigt auch einen längeren Endausfall. Abbildung 4.4 fasst den ergänzten Anteil innerhalb der aktiven Zeit jedes Sensors zusammen.

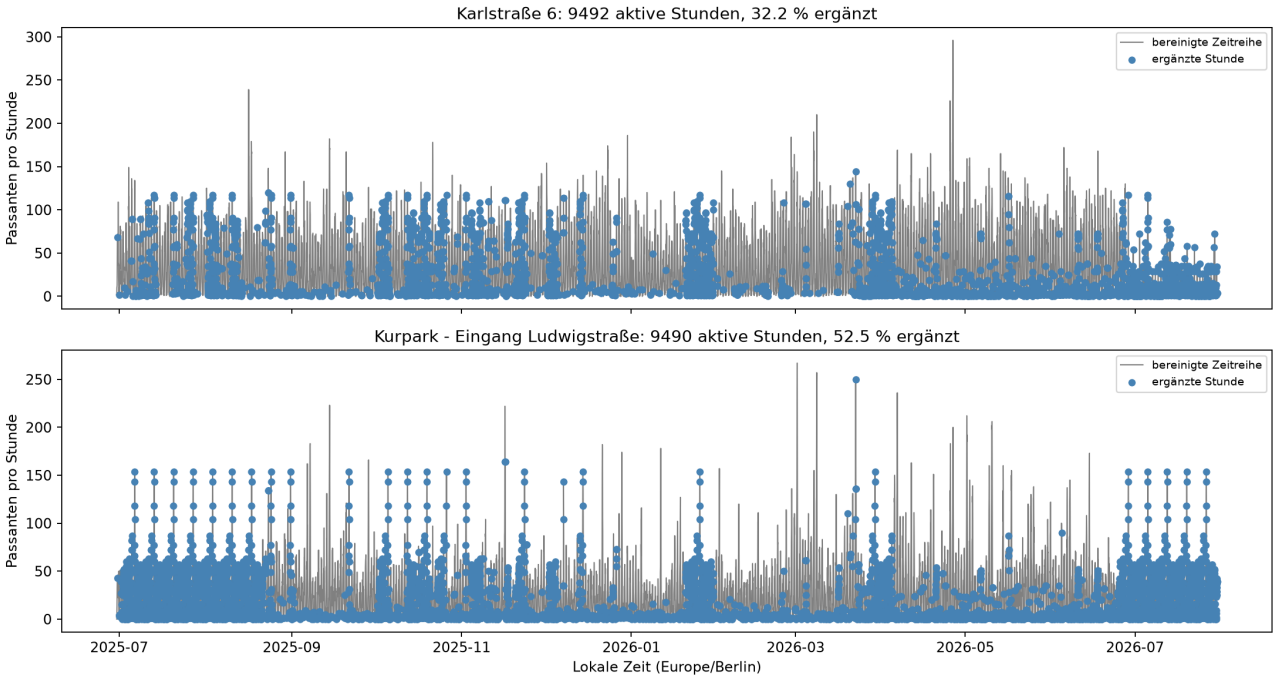


Abbildung 4.3: Bereinigte Zeitreihen eines typischen Sensors (oben) und des Sensors mit dem höchsten Imputationsanteil (unten). Markierte Punkte sind ergänzte Stunden. Eigene Darstellung.

4.5 Kritische Betrachtung der Bereinigung

Jede Imputation verändert die Daten. Mittelwertbasierte Verfahren können den Verlauf glätten und die Varianz verringern [25, 17]. Im vorliegenden Datensatz ändert sich der Mittelwert der Passantenzahl je Sensor um höchstens 22,88 %. Die kleinste Korrelation zwischen dem ausschließlich aus gemessenen Werten und dem aus allen bereinigten Werten gebildeten stündlichen Tagesprofil beträgt 0,9973. Diese Kennzahlen prüfen Niveau und Profil, ersetzen aber keine Fehleranalyse gegen unbekannte wahre Werte in den Lücken.

Die zweistündige Fortschreibung verwendet zu jedem Zeitpunkt nur die Vergangenheit. Die statistischen Profile dürfen dagegen innerhalb des Trainingsabschnitts auch chronologisch spätere Trainingsmessungen enthalten. Die Aufbereitung ist damit für die Out-of-Sample-Auswertung frei von Validierungs- und Testdaten-Leakage, jedoch nicht als rein online-kausale Imputation jeder frühen Trainingslücke zu verstehen.

Die erste vorhandene Messung dient mangels eines Betriebsplans als vorläufiger Beginn der Sensoraktivität. Danach wird ein Sensor bis zum globalen Datenende als aktiv behandelt, damit ein dauerhaft ausbleibender Stream nicht fälschlich verschwindet. Geplante Abschaltungen können dadurch vorläufig wie Ausfälle aussehen. Die vorgesehenen Aktivitätszeiträume müssen deshalb noch mit den Betreiberdaten abgeglichen werden. Die Datei `data/sensor_activity_periods.csv` hält dafür beobachtete Grenzen und noch offene Felder für die geplanten Betriebszeiten fest.

Ein zweiter Aspekt betrifft die Anomalieerkennung selbst. Ergänzte Stunden wirken per Konstruktion un-

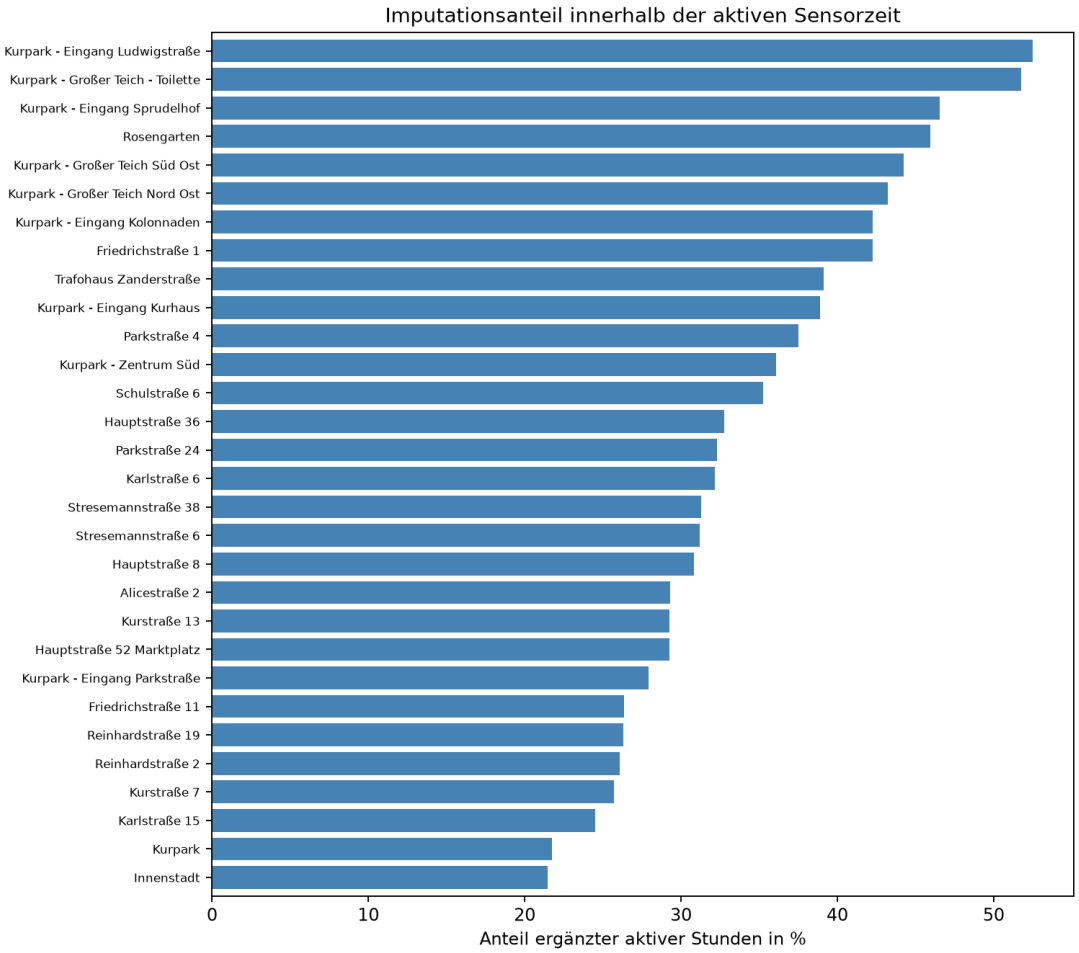


Abbildung 4.4: Anteil ergänzter Stunden innerhalb der aktiven Zeit jedes Sensors. Eigene Darstellung.

auffällig, ein tatsächlicher Sensorausfall stellt jedoch selbst ein meldenswertes Ereignis dar. Das Fehlen von Werten folgt hier zudem eigenen, teils blockweisen Mustern und trägt damit selbst Information [25]. Das Feld `is_imputed` hält für jede Stunde fest, ob sie gemessen oder ergänzt wurde. Beide Prognosemodelle verwenden ergänzte Werte weiterhin als Kontext eines 24-Stunden-Fensters, überspringen jedoch Fenster mit einem ergänzten Zielwert beim Training, bei der Bestimmung des Schwellenwerts und bei der Testauswertung. Alle 30 Sensorreihen erfüllen im Jahresdatensatz die Mindestmenge gemessener Trainingsziele.

4.6 Forecast-basierter Anomaliedetektor

4.6.1 Lernaufgabe und Eingaberepräsentation

Der Forecaster lernt selbstüberwacht, aus den jeweils vorhergehenden 24 Stunden die nächste Stunde vorherzusagen. Die nachgelagerte Anomalieentscheidung ist unüberwacht, da weder das Modelltraining noch die Kalibrierung reale Anomalieklassen verwenden. Ein einziges globales Modell wird über alle 30 Sensoren trainiert. Jeder Sensor besitzt vor dem Ende des Trainingsabschnitts mindestens 72 gemessene Zielstunden. Fehlende Stunden werden unabhängig davon für alle Sensoren als Datenqualitätsereignisse ausgegeben.

Beide Zielgrößen werden zunächst mit $\log(1 + x)$ transformiert. Danach erfolgt eine robuste Skalierung je

Sensor s und Merkmal f:

$$z_{s,t,f} = \frac{\log(1 + x_{s,t,f}) - \text{Median}_{s,f}}{\max(\text{IQR}_{s,f}, 0{,}05)}. \tag{4.2}$$

Median und Interquartilsabstand werden ausschließlich aus gemessenen Trainingswerten bestimmt. Die sensorweisen Mittelwerte der Passantenzahl reichen in der Analyse von etwa 12,3 bis 259,0 Personen pro Stunde; die sensorweise Skalierung verhindert, dass diese Niveauunterschiede die Modellierung dominieren. Jedes historische Zeitfenster besitzt acht Kanäle: die beiden skalierten Messgrößen, eine Maske für gemessen beziehungsweise imputiert sowie Sinus und Kosinus von lokaler Stunde und Wochentag und einen Wochenendindikator. Die fünf Kalendermerkmale der Zielstunde und eine Sensor-ID werden dem Modell zusätzlich übergeben. Beide Messgrößen unterstützen damit das Lernen des zeitlichen Kontexts. Das fachliche Anomalieziel des Projekts ist jedoch ausschließlich die Passantenfrequenz.

4.6.2 Architektur und Größenwahl

Die Architektur übernimmt aus den Literaturbefunden in Abschnitt 3.1 Quantilausgaben, ein kleines Sensor-Embedding, Kalendermerkmale und Dropout. Der Kern bleibt eine einzelne LSTM-Schicht.

Das LSTM verdichtet das 24-Stunden-Fenster in einen Zustandsvektor [4]. Dieser wird mit einem lernbaren Sensor-Embedding und den Kalendermerkmalen der Zielstunde verbunden und durch eine dichte Schicht geführt. Das globale Training und das Sensor-Embedding orientieren sich an probabilistischen globalen RNN-Forecastern [14]; aufgrund der Quantilausgabe handelt es sich jedoch nicht um eine DeepAR-Implementierung.

Das Ausgabefeld enthält für `visitors` und `avgDuration` jeweils die Quantile $q_{0,1}$, $q_{0,5}$ und $q_{0,9}$. Trainiert wird mit der über beide Zielgrößen und Quantile gemittelten Pinball-Loss-Funktion [8]. Fünf vorab festgelegte Kandidaten variieren LSTM- und Dense-Breite, Embedding, Dropout und Lernrate. Der kleinste Tuning-Loss entscheidet, ohne den Testabschnitt zu verwenden. Tabelle 4.2 zeigt die ausgewählte Konfiguration. Die Prognose der Verweildauer bleibt eine Hilfsaufgabe; nur die `visitors`-Ausgabe wird später zur Alarmbildung verwendet.

Tabelle 4.2: Ausgewählte Forecaster-Architektur. Eigene Darstellung.

Baustein	Größe	Parameter	Begründung für diesen Datensatz
LSTM	64	18.688	Verdichtet ein Fenster aus 24 Stunden und acht Kanälen. Eine einzelne rekurrente Schicht hält die Architektur trotz 148.045 Trainingszielen kompakt.
Sensor-Embedding	4	120	Bildet 30 Sensoren mit $30 \cdot 4$ Werten als kleine Zusatzinformation ab.
Dense	32	2.368	Mischt den 73-dimensionalen Vektor aus LSTM, Embedding und fünf Kalenderwerten und verkleinert ihn auf 32 Werte.
Dropout	0,10	0	Begrenzt die Anpassung an den kurzen und durch überlappende Fenster stark ähnlichen Trainingszeitraum.
Ausgabe	6	198	Durch zwei Messgrößen und drei Quantile je Messgröße vollständig vorgegeben.

Insgesamt besitzt das ausgewählte Modell 21.374 trainierbare Parameter. Fünf Kombinationen reichen von 16 bis 128 LSTM-Einheiten und variieren zusätzlich Dense-Breite, Embedding, Dropout und Lernrate. Nach zunächst zehn Epochen reduziert Successive Halving die Menge von fünf auf drei und dann zwei Finalisten. Beide Finalisten laufen warm gestartet mit einer Obergrenze von 80 Epochen und Early-Stopping-Geduld acht. Die ausgewählte Konfiguration wird für ihre beste Epochenzahl, hier 73, mit Batchgröße 512 auf Training und Tuning neu trainiert; Kalibrierungs- und Testdaten bleiben dabei unberührt.

4.6.3 Nichtkonformitätsscore und Intervallkalibrierung

Für Merkmal f wird die Abweichung einer Beobachtung $y_{t,f}$ vom vorhergesagten Quantilintervall relativ zu dessen Breite bewertet:

$$e_{t,f} = \frac{\max\{q_{0,1,t,f} - y_{t,f},\, y_{t,f} - q_{0,9,t,f}\}}{\max\{q_{0,9,t,f} - q_{0,1,t,f},\, w_{\min,f}\}}, \qquad s_t = e_{t,\text{visitors}}. \tag{4.3}$$

Ein negativer Wert liegt innerhalb des ursprünglichen Quantilintervalls; ein positiver Wert misst die relative Überschreitung. Die Fehler beider Größen werden zur Diagnose ausgegeben; allein der Besucherfehler s_t steuert den Alarm. Aus 13.142 gemessenen Punkten des getrennten Kalibrierungsabschnitts wird bei nominal $\alpha = 0{,}01$ eine endliche empirische Quantilschwelle bestimmt. Das Verfahren ist an Conformalized Quantile Regression angelehnt [13] und verwendet zusätzlich eine Breitenstandardisierung. Die Intervalle beider Größen werden getrennt kalibriert; die Alarmschwelle beruht nur auf `visitors`. Wegen der zeitlichen Abhängigkeit der Fenster wird keine verteilungsfreie 99-%-Garantie beansprucht [1]; die tatsächliche Testabdeckung wird berichtet.

4.6.4 Kausale Alarmregel

Starke Einzelabweichungen werden unmittelbar erkannt. Als mögliche Erweiterung für moderate, aber anhaltende Abweichungen wird auch eine kausale Persistenzregel geprüft:

$$A_t = \mathbf{1}(s_t > \tau_{\text{hoch}}) \;\vee\; \mathbf{1}\left(\sum_{j=0}^{m-1} \mathbf{1}(s_{t-j} > \tau_{\text{niedrig}}) \geq k\right). \tag{4.4}$$

Eine solche Regel arbeitet je Sensor, betrachtet nur aktuelle und vergangene Scores und wird bei einer Zeitlücke oder einem imputierten Ziel zurückgesetzt. Aus 18 vorab festgelegten Kombinationen werden τ_{hoch}, τ_{niedrig}, m und k ausschließlich anhand synthetischer Anomalien im Kalibrierungsabschnitt gewählt. Zulässig sind nur Regeln mit höchstens 1 % Alarmen auf den unveränderten Kalibrierungswerten. Optimiert wird der über die Anomalietypen gleichgewichtete Ereignis-F1; die Injektionen des Testabschnitts beeinflussen diese Auswahl nicht.

4.6.5 Umgang mit imputierten Zielwerten

Imputierte Werte dürfen als markierter Kontext in ein Zeitfenster eingehen, werden aber nicht als Ziel für Training, Modellauswahl, Kalibrierung oder Forecast-Metriken verwendet. Eine fehlende Originalmessung wird unabhängig vom Forecast als Datenqualitätsereignis ausgegeben. Dadurch wird ein aus einem Zeitprofil erzeugter Ersatzwert weder als reguläre Beobachtung noch als bestätigte Verhaltensanomalie fehlinterpretiert. Ein Forecast-Alarm wird nicht nachträglich unterdrückt, erhält aber niedrige Konfidenz, wenn mehr als die Hälfte seines 24-Stunden-Kontexts imputiert ist oder die Besucherzahl die Intervallgrenze um weniger als eine Person überschreitet. Der jeweilige Grund wird in der Kandidatendatei ausgegeben.

4.7 Transformer-Forecaster

Der Transformer löst dieselbe selbstüberwachte Prognoseaufgabe wie der LSTM-Forecaster. Er erhält dieselben 24 historischen Stunden, Sensor-IDs, Kalendermerkmale und Zielgrößen. Auch Skalierung, zeitliche Teilmengen, Quantile, Nichtkonformitätsscore und Alarmkalibrierung bleiben unverändert. Dadurch prüft der spätere Vergleich die Architektur und nicht unterschiedliche Daten- oder Bewertungsregeln.

Jede Stunde bildet ein eigenes Token. Eine Dense-Projektion überführt dessen acht Eingabekanäle in 48 Merkmale; eine feste sinusförmige Positionskodierung erhält die Reihenfolge. Das achtdimensionale Sensor-Embedding wird auf dieselbe Breite projiziert und jedem Token hinzugefügt. Anschließend folgen zwei

Pre-LN-Encoderblöcke mit jeweils vier Attention-Köpfen, einem Feedforward-Netz der Breite 192, GELU-Aktivierung, Residualpfaden und 0,15 Dropout. Pre-LN folgt Xiong et al.; Attention, Residualpfade und Positionskodierung folgen dem ursprünglichen Transformer [23, 21].

Der Zustand des letzten Tokens wird mit dem Sensor-Embedding und den fünf bekannten Kalenderwerten der Zielstunde verbunden. Eine Dense-Schicht der Breite 48 erzeugt daraus die Prognose. Parallel bildet ein linearer Pfad die 24 historischen Werte beider Messgrößen direkt auf die zwei Mediane ab. Dieser einfache autoregressive Anteil ist durch die Ergebnisse linearer Zeitreihenmodelle motiviert [24]. Die beiden übrigen Ausgabewerte werden mit Softplus als nichtnegative Abstände vom Median dargestellt. Damit gilt konstruktiv $q_{0,1} \leq q_{0,5} \leq q_{0,9}$; ein nachträgliches Sortieren der Quantile ist nicht erforderlich.

Tabelle 4.3: Ausgewählte Transformer-Architektur. Eigene Darstellung.

Baustein	Größe	Parameter	Begründung für diesen Datensatz
Eingabeprojektion	48	432	Bewahrt alle 24 Stunden als einzelne, kompakte Tokens.
Sensorkontext	$8 \rightarrow 48$	624	Embedding und Projektion ergänzen standortspezifische Information in jedem Token.
Zwei Pre-LN-Blöcke	4 Köpfe, FFN 192	56.544	Vier Köpfe ergeben bei Modellbreite 48 je zwölf Merkmale; zwei Blöcke erlauben eine zweite zeitliche Verknüpfung, ohne das Parameterbudget stark zu erhöhen.
Prognosekopf	Dense 48	3.366	Verbindet letzten Token, Sensor und Zielkalender und erzeugt sechs Quantilwerte.
Linearer Pfad	$48 \rightarrow 2$	96	Hält einfache autoregressive Zusammenhänge direkt erreichbar.

Insgesamt besitzt das Modell 61.062 trainierbare Parameter. Sechs vorab festgelegte Kandidaten mit 3.074 bis 156.886 Parametern variieren Modellbreite, Kopf- und Blockzahl, Feedforward- und Dense-Breite, Embedding, Dropout und Lernrate. Successive Halving reduziert sie nach jeweils zehn und 20 Epochen auf zwei Finalisten. Diese laufen warm gestartet bis höchstens 120 Epochen mit Early-Stopping-Geduld zwölf. Trainiert wird mit AdamW, Gewichtsnachlass 10^{-4}, auf eins begrenzter Gradientennorm und Batchgröße 512. Der Tuning-Loss entscheidet wie beim LSTM ohne Nutzung von Kalibrierungs- oder Testdaten. Patching wird nicht verwendet: Sein Vorteil betrifft vor allem lange Eingaben, während das 24-Stunden-Fenster kurz ist und einzelne Stundensprünge erhalten soll [12].

4.8 Denoising-Autoencoder

Der Autoencoder bildet eine von den Prognosemodellen getrennte Methode. Er erhält ein 24-Stunden-Fenster mit den beiden skalierten Messgrößen und der Beobachtungsmaske. Seine Ausgabe rekonstruiert die beiden Messgrößen für alle 24 Stunden. Der vollständige Aufbau lautet:

$$\mathrm{Input}(24, 3) \rightarrow \mathrm{LSTM}(64) \rightarrow \mathrm{RepeatVector}(24) \rightarrow \mathrm{LSTM}(64) \rightarrow \mathrm{Dense}(2). \tag{4.5}$$

Die einzelne LSTM-Schicht auf beiden Seiten des komprimierten Zustands folgt dem Grundaufbau von Malhotra et al. [11]. Die Literatur bestimmt damit die Form, während Tabelle 4.4 die an den vorliegenden Datensatz angepassten Größen begründet.

Das Netz besitzt insgesamt 50.562 trainierbare Parameter. Fünf vorab festgelegte Kandidaten variieren LSTM-Breite, Lernrate und maximale Maskierungslänge. Successive Halving reduziert sie nach zehn und 20 Epochen auf zwei vollständig trainierte Finalisten. Der Tuning-Loss wählt 64 Einheiten, eine Lernrate von 0,0003 und höchstens sechs maskierte Stunden. Eine zusätzliche versteckte Dense-Schicht ist nicht nötig, da die vorhandene Rekonstruktion bereits beide Messgrößen direkt aus dem Decoderzustand erzeugt.

Während des Trainings wird in jedem Fenster ein zusammenhängender Block von einer bis sechs wirklich gemessenen Stunden verdeckt. Damit bleiben selbst im größten Fall 18 der 24 Stunden als zeitlicher Kontext

Tabelle 4.4: Größenwahl des Autoencoders. Eigene Darstellung.

Baustein	Größe	Parameter	Begründung für diesen Datensatz
Eingabe	24×3	0	Ein Tag mit zwei Messgrößen und einer Beobachtungsmaske.
Encoder-LSTM	64	17.408	Verdichtet das 24-Stunden-Fenster auf 64 Zustandswerte.
RepeatVector	24	0	Stellt wieder genau die ursprüngliche Länge von 24 Stunden bereit.
Decoder-LSTM	64	33.024	Verwendet dieselbe Breite wie der Encoder und führt dadurch keine weitere frei gewählte Größe ein.
Dense	2	130	Durch die Rekonstruktion von `visitors` und `avgDuration` vollständig vorgegeben.

erhalten. Die zugehörigen Eingabewerte und ihre Maske werden auf null gesetzt, das Rekonstruktionsziel bleibt jedoch unverändert. Diese einfache Datenerweiterung folgt dem Denoising-Prinzip [22]. Künstliche Spikes, Dips oder andere Testanomalien werden nicht für das Training verwendet.

Trainiert wird mit Adam, Batchgröße 512, höchstens 200 Epochen und Early-Stopping-Geduld 15. Die Huber-Fehlerfunktion verringert den Einfluss einzelner großer, im unbeschrifteten Training möglicherweise bereits vorhandener Abweichungen. Ergänzte Werte dürfen als markierter Kontext eingehen, besitzen im Rekonstruktionsfehler aber das Gewicht null. Das Modell lernt sie daher nicht als echte Zielwerte.

Für jede Stunde wird der absolute Rekonstruktionsfehler der letzten Position des Fensters verwendet. Der Besucherfehler wird durch sein 75-%-Quantil im Kalibrierungsabschnitt geteilt und bildet den Anomaliewert. Die Rekonstruktion von `avgDuration` bleibt eine Hilfsaufgabe und wird zur Diagnose ausgegeben, kann aber keinen Alarm auslösen. Die Alarmregel wird anschließend mit demselben Kalibrierungsverfahren wie beim Forecaster festgelegt. Dadurch unterscheiden sich die Modelle, während Datenabschnitte, Besucher-Injektionen und Entscheidungskriterien gleich bleiben.

4.9 Robuste saisonale Baseline

Als einfachste Vergleichsmethode wird kein weiteres Netz trainiert. Für jede gemessene Stunde wird der sensorweise robust skalierte Besucherwert von dem ebenfalls gemessenen Wert 24 Stunden zuvor abgezogen. Median und Interquartilsabstand dieser Residuen werden nur aus gemessenen Trainingspaaren bestimmt; ihr standardisierter Absolutwert bildet den Score. Die Baseline besitzt damit keine trainierbaren Parameter. Sie verwendet dieselbe Kalibrierung, dieselben Besucher-Injektionen und dieselben Entscheidungskriterien wie die neuronalen Modelle. Sie liefert nur für Stunden mit einem gemessenen $t - 24$-Vergleichswert einen Score und enthält sich an den übrigen Stunden eines Alarms.

Kapitel 5

Evaluation der Methode

5.1 Zeitliche Datentrennung

Alle Teilmengen werden chronologisch und nicht zufällig gebildet. Dies verhindert, dass spätere Stunden als Lernbeispiele für frühere Prognosen verwendet werden. Tabelle 5.1 nennt die Grenzen und die Anzahl tatsächlich gemessener Zielpunkte nach dem 24-Stunden-Windowing.

Tabelle 5.1: Chronologische Teilmengen aller Verfahren. Alle Grenzen sind in UTC angegeben. Eigene Darstellung.

Teilmenge	Beginn	Ende (exklusiv)	Zielpunkte
Training	erster möglicher Zielwert	01.05.2026	148.045
Tuning	01.05.2026	01.06.2026	15.159
Kalibrierung	01.06.2026	01.07.2026	13.142
Test	01.07.2026	Reihenende	9.604

Für LSTM-Forecaster und Autoencoder werden jeweils fünf, für den Transformer sechs vorab festgelegte Architektur- und Lernratenkombinationen verglichen. Die Kandidaten decken bewusst kleine bis deutlich größere Netze ab. Jeder Kandidat lernt nur auf dem Trainingsabschnitt; ausschließlich der chronologische Tuning-Loss bestimmt Kombination und Epochenzahl. Um die Suche zu verkürzen, erhalten zunächst alle Kandidaten zehn Epochen. Danach wird die schlechtere Hälfte verworfen, bis zwei Finalisten verbleiben. Diese beiden werden warm gestartet bis zur jeweiligen Obergrenze mit Early Stopping weitertrainiert und erst anschließend verglichen. Damit kann ein in der Finalrunde später aufholendes Modell noch gewinnen; eine absolute Garantie gegen das zu frühe Verwerfen eines anderen Kandidaten böte allerdings nur die wesentlich teurere Vollsuche.

Der LSTM-Forecaster erhält höchstens 80 Epochen bei einer Geduld von acht, der Transformer 120 bei einer Geduld von zwölf und der Autoencoder 200 bei einer Geduld von 15 Epochen. Die Batchgröße beträgt einheitlich 512. Das ausgewählte Modell wird anschließend für seine beste Epochenzahl auf Training und Tuning gemeinsam, also auf 163.204 Fenstern mit identischen Zielzeitpunkten, neu trainiert. Erst danach bestimmt der Kalibrierungsabschnitt die Schwellen und gegebenenfalls eine Persistenzregel. Der Testabschnitt wird für keine dieser Entscheidungen verwendet.

5.2 Bewertung der Prognosegüte

Die Medianprognose $q_{0,5}$ beider Prognosemodelle wird mit MAE und RMSE bewertet. Zusätzlich wird die saisonal skalierte mittlere absolute Abweichung (MASE) nach Hyndman und Koehler berichtet [6]:

$$\mathrm{MASE} = \frac{H^{-1}\sum_{h=1}^{H}|y_{T+h} - \hat{y}_{T+h}|}{(T-24)^{-1}\sum_{t=25}^{T}|y_t - y_{t-24}|}. \tag{5.1}$$

Der Nenner wird ausschließlich aus Paaren berechnet, bei denen sowohl das Trainingsziel als auch sein $t-24$-Vergleichswert gemessen wurden. Damit ist MASE von dem ergänzend angegebenen Verhältnis $\mathrm{MAE}_{\mathrm{Modell,Test}} \,/\, \mathrm{MAE}_{t-24,\mathrm{Test}}$ zu unterscheiden. Werte unter eins bedeuten jeweils eine Verbesserung gegenüber der zugehörigen Skalierung beziehungsweise Testbaseline. Auch der direkte Testvergleich verwendet nur die 8.382 Testpunkte mit gemessenem aktuellem Wert und gemessenem $t-24$-Wert.

Für die empirisch kalibrierten Intervalle werden die marginale Abdeckung je Zielgröße, die gemeinsame Abdeckung beider Zielgrößen und die mittlere Intervallbreite ausgewiesen. Eine hohe Abdeckung allein ist nicht hinreichend, da sie durch sehr breite und damit wenig trennscharfe Intervalle erreichbar wäre. Diese Werte beschreiben die Prognosen beider Größen. Für die Alarmentscheidung wird ausschließlich der kalibrierte Besucher-Score verwendet.

Als einfache robuste Vergleichsmethode wird zusätzlich die Abweichung vom gemessenen Wert derselben Stunde des Vortags verwendet. Nach der sensorweisen robusten Skalierung werden die Residuen durch Median und Interquartilsabstand der gemessenen Trainingspaare standardisiert; Schwelle und Persistenzregel werden anschließend genauso wie bei den drei neuronalen Modellen kalibriert. Der Detektor besitzt keine trainierbaren Parameter und kann nur Punkte bewerten, für die der aktuelle Wert und der $t-24$-Wert tatsächlich gemessen wurden. Fehlt der Vergleichswert, enthält sich die Baseline eines binären Alarms; ein dort injizierter Punkt bleibt damit gegebenenfalls ein False Negative.

5.3 Synthetische Anomalieinjektion

Reale Anomalieklassen liegen nicht vor. Deshalb wird die technische Erkennungsfähigkeit ergänzend mit kontrollierten Injektionen geprüft. Dieses Vorgehen liefert eindeutige Labels für genau definierte Fehlertypen, ersetzt aber keine Evaluation realer Ereignisse [16]. Die Anomalien werden ausschließlich in ursprünglich gemessene Werte injiziert, bevor Zeitfenster erzeugt und Prognosen berechnet werden. Damit beeinflussen mehrstündige Ereignisse den nachfolgenden Modellkontext wie reale Messwerte. Alle vier Detektoren erhalten dieselben ausgewählten Sensoren, Zeitpunkte, Richtungen und Stärken. Verändert wird ausschließlich `visitors`, weil die gesuchte Anomalie eine Abweichung der Passantenfrequenz ist.

Untersucht werden ein einstündiger positiver Spike, ein einstündiger Dip, ein vierstündiger Level-Shift, ein über vier Stunden ansteigender Drift und ein sechsstündiger Stuck-at-Fehler. Bei den mehrstündigen Typen werden beide Richtungen abgedeckt. Die Stärken 0,5, 1,0 und 2,0 beziehen sich auf sensor- und besucherspezifische robuste $\log(1+x)$-IQR-Einheiten. Stuck-at friert die Passantenzahl auf die letzte vorherige Messung ein; Ereignisse mit weniger als 0,5 realisierter IQR-Abweichung werden verworfen, weil sie keine belastbare künstliche Veränderung darstellen. Ein Punkt erhält erst nach Rundung und Begrenzung auf nichtnegative Zählwerte ein Label, wenn sich sein Wert tatsächlich geändert hat. Wirkungslos gewordene Dips und Stuck-at-Ereignisse werden dadurch nicht fälschlich als positive Beispiele bewertet.

Die Auswahl der Alarmregel verwendet drei feste Zufallsstartwerte im Kalibrierungsabschnitt und nur die Stärken 1,0 und 2,0. Der endgültige Vergleichstest verwendet drei andere feste Zufallsstartwerte im Testabschnitt und schließt 0,5 als Sensitivitätsanalyse ein. Pro Lauf wird höchstens ein Ereignis je Sensor erzeugt. Jede injizierte Reihe wird mit einem identischen unveränderten Gegenlauf gepaart. Ein Treffer zählt nur, wenn die Injektion einen zusätzlichen Alarm hervorruft; ein bereits am gleichen Zeitpunkt bestehender Alarm wird nicht nachträglich als Erfolg der Injektion gewertet.

5.4 Punkt- und Ereignismetriken

Auf Punktebene werden aus den binären zusätzlichen Alarmen Konfusionsmatrix, Precision, Recall, F1 und zeitliche Intersection over Union berechnet. Die Konfusionswerte werden zunächst über alle Läufe summiert und erst danach in Kennzahlen umgerechnet. ROC-AUC und Average Precision (AP) bewerten davon getrennt die kontinuierlichen Rohscores; eine binäre Alarmentscheidung wird nicht als Rankingkurve ausgegeben. Für eine gemeinsame Testpopulation werden fehlende Baseline-Scores bei den Rankingmetriken unterhalb aller endlichen Scores eingeordnet und ihre Anzahl separat gespeichert. Precision und Recall sind durch

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{5.2}$$

definiert. AP ist wegen des starken Ungleichgewichts zwischen Normal- und Anomaliepunkten besonders informativ; ROC-AUC wird ergänzend ausgegeben.

Zusammenhängende stündliche Alarme desselben Sensors werden zusätzlich als Ereignisse gruppiert. Ein vorhergesagtes Ereignis darf höchstens einem injizierten Ereignis zugeordnet werden; Fragmentierung kann dadurch den Ereignis-F1 nicht künstlich erhöhen. Berichtet werden Ereignis-Precision, Ereignis-Recall, Ereignis-F1, mittlere zeitliche IoU und die Verzögerung bis zum ersten überlappenden Alarm. Diese Existenz- und Überlappungsdefinition ist eine bewusst einfache ereignisorientierte Auswertung und nicht das vollständige Metriksystem von Tatbul et al. [20]. Die ursprünglichen Punktvorhersagen werden nicht verändert. Insbesondere wird kein Point Adjustment verwendet, da es die gemessene Leistung stark überschätzen kann [7].

Ergänzend prüft ein reproduzierbarer Rauschtest die Alarmstabilität. Für gemessene Besucherwerte wird bei 5 % und 15 % Stärke mittelwerttreues, nichtnegatives Zählwertrauschen aus binomialem Ausdünnen und einer Poisson-Ersetzung erzeugt. Berichtet wird der Anteil zusätzlicher Alarme gegenüber dem unveränderten Testlauf. Der einmonatige Testblock wird nicht noch einmal in kürzere Teilblöcke zur Rolling-Origin-Neuschätzung zerlegt, weil dann pro Ursprung nur wenige Ereignisse und Sensorstunden verblieben.

5.5 Reproduzierbarkeit

Die explorative Datenqualitätsanalyse ist in `analyse/analyse.py` erhalten; `analyse/preprocessing_plots.py` erzeugt die Diagnoseabbildungen zur Aufbereitung. Die eigentliche Pipeline wird aus dem Projektverzeichnis mit folgenden Befehlen ausgeführt:

```
py -3.12 -m src.download_data
py -3.12 -m src.preprocessing
py -3.12 -m src.forecaster.train
py -3.12 -m src.transformer.train
py -3.12 -m src.autoencoder.train
py -3.12 -m src.seasonal_baseline
py -3.12 -m src.benchmark
py -3.12 -m src.forecaster.evaluate
py -3.12 -m src.forecaster.evaluate --model-dir saved_models/transformer
```

Bei unveränderten Rohdaten werden nur die gewünschten Modelle neu trainiert. Für die erneute Auswertung des gespeicherten LSTM-Forecasters reicht `py -3.12 -m src.forecaster.evaluate`; derselbe Aufruf mit `--model-dir saved_models/transformer` wertet den Transformer aus. Der davon getrennte synthetische Vergleich aller vier Verfahren wird mit `py -3.12 -m src.benchmark` ausgeführt. Modellparameter, Splits, Skalierung, Kalibrierung und Detektorregel werden in `model_bundle.json` gespeichert. Die

Auswertung erzeugt unter anderem `evaluation_metrics.json`, `anomalies.csv`, `anomaly_events.csv`, `labels_template.csv`, `event_review_template.csv` und die Abbildung `anomalies_example.png`. Der gemeinsame Injektionstest schreibt die Detailergebnisse je Modell und fasst alle vier Verfahren zusätzlich in `saved_models/benchmark/model_comparison.csv` sowie in der gemeinsamen ROC-/PR-Abbildung zusammen. Der zugehörige Code ist nach Injektionen, Metriken, Auswertung und der kurzen Aufrufsteuerung in getrennte Module gegliedert.

Kapitel 6

Ergebnisse

6.1 Training und Detektorkalibrierung

Beim LSTM-Forecaster erreicht Kandidat 3 aus fünf Konfigurationen den kleinsten Tuning-Loss. Er verwendet 64 LSTM-, 32 Dense- und vier Embedding-Einheiten, 0,10 Dropout und eine Lernrate von 0,001. Sein Tuning-Loss beträgt 0,1158 bei Epoche 73. Das endgültige Modell wird daher 73 Epochen auf 163.204 gemessenen Zielen aus Training und Tuning neu trainiert und besitzt 21.374 Parameter. Kalibrierungs- und Testabschnitt werden dabei nicht zum Anpassen der Modellgewichte verwendet. Abbildung 6.1 zeigt den Trainingsverlauf.

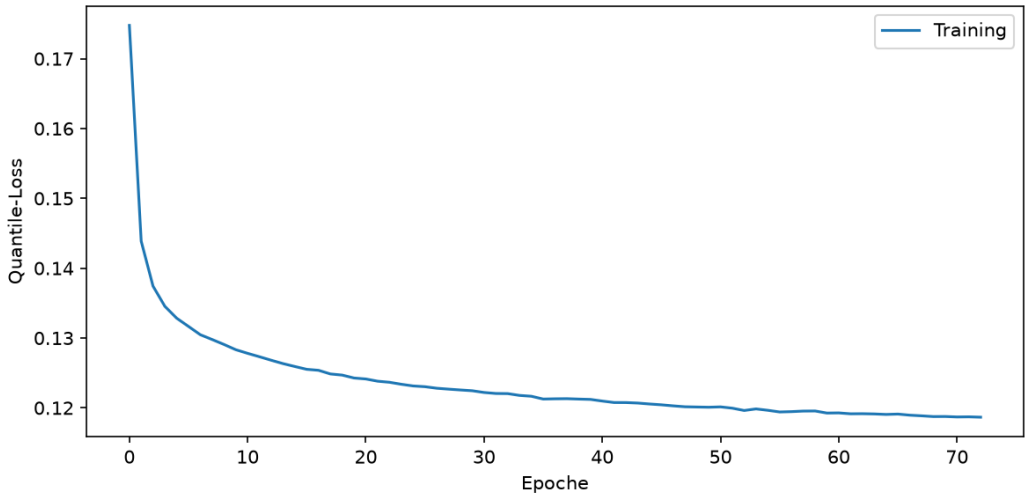


Abbildung 6.1: Pinball-Loss des endgültigen Forecaster-Trainings. Eigene Darstellung.

Beim Transformer gewinnt Kandidat 4 aus sechs Konfigurationen mit Modellbreite 48, vier Attention-Köpfen, zwei Encoderblöcken, Feedforward-Breite 192 und 0,15 Dropout. Sein Tuning-Loss beträgt 0,1097. Der Finalistenlauf endet nach 82 Epochen; der beste Wert liegt bei Epoche 70. Das für 70 Epochen neu trainierte Modell besitzt 61.062 Parameter. Abbildung 6.2 zeigt den zugehörigen Verlauf.

Beim Autoencoder gewinnt Kandidat 4 mit jeweils 64 LSTM-Einheiten in Encoder und Decoder, einer Lernrate von 0,0003 und höchstens sechs zusammenhängend maskierten Stunden. Beide Finalisten schöpfen die Obergrenze von 200 Epochen aus. Der gewählte Kandidat erreicht dort einen Tuning-Loss von 0,0316 und besitzt 50.562 Parameter. Dieser Rekonstruktionsloss ist wegen einer anderen Zielfunktion nicht direkt mit den Pinball-Loss-Werten der Forecaster vergleichbar.

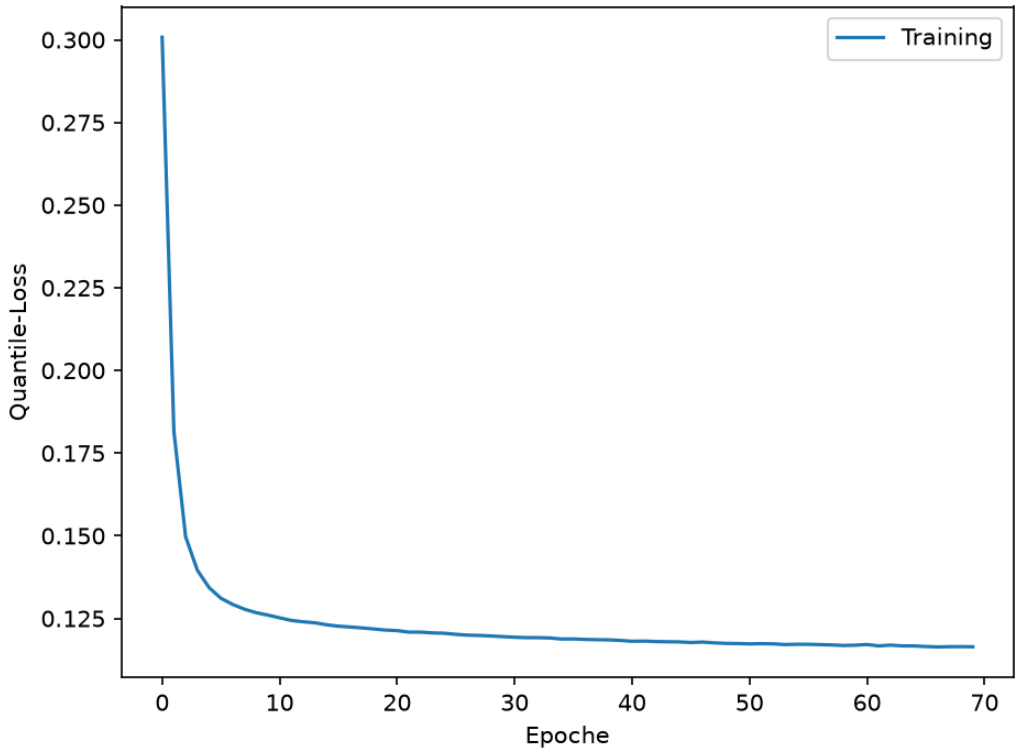


Abbildung 6.2: Pinball-Loss des endgültigen Transformer-Trainings. Eigene Darstellung.

Die Kalibrierung des LSTM wählt eine hohe Besucher-Schwelle von 1,2455 und eine kausale Zwei-von-drei-Regel oberhalb der niedrigeren Schwelle 0,3342. Beim Transformer betragen diese Schwellen 1,4101 und 0,4000; auch dort wird die Zwei-von-drei-Regel gewählt. Auf den 13.142 unveränderten Kalibrierungspunkten erzeugen die Regeln 120 beziehungsweise 115 Alarme, also 0,913 % beziehungsweise 0,875 %, und halten das vorab festgelegte 1-%-Budget ein. Der Macro-Ereignis-F1 der Kalibrierungsinjektionen beträgt 0,856 für den LSTM und 0,816 für den Transformer. Diese Werte sind Auswahlkriterien und keine Testergebnisse. 1.497 Kalibrierungsfenster (11,39 %) bestehen zu mehr als der Hälfte aus imputierten Kontextstunden; die Mehrheit der Kalibrierungskontexte ist damit ausreichend beobachtet.

6.2 Prognosegüte im Testabschnitt

Tabelle 6.1: Prognosegüte auf 9.604 gemessenen Testzielen. Die Abdeckung und Breite beziehen sich auf die empirisch kalibrierten Intervalle. Eigene Darstellung.

Modell	Ziel	MAE	RMSE	MASE	rel. MAE	Abdeckung	Breite
LSTM	Visitors	11,796	19,338	0,663	0,671	98,90 %	160,75
Transformer	Visitors	11,166	18,381	0,628	0,637	99,10 %	167,02
LSTM	AvgDuration	3,054	5,550	0,693	0,726	98,97 %	35,53
Transformer	AvgDuration	2,972	5,494	0,675	0,709	98,97 %	41,94

Alle MASE-Werte liegen unter eins. Der Transformer erzielt bei beiden Zielgrößen kleinere MAE- und RMSE-Werte als der LSTM. Auf den 8.382 vollständig gemessenen $t-24$-Paaren verbessert er den Besucher-MAE um etwa 36,3 % gegenüber der saisonalen Prognose; beim LSTM sind es etwa 32,9 %. Die gemeinsame empirische Abdeckung beider Zielgrößen beträgt 98,59 % beim Transformer und 98,46 % beim LSTM. Die

Transformer-Intervalle sind dabei etwas breiter. Abdeckung und Breite sind daher gemeinsam zu interpretieren und belegen allein keine hohe Trennschärfe des Anomaliedetektors. Die Werte für `avgDuration` und die gemeinsame Abdeckung sind Prognosediagnosen; der Alarm verwendet nur den Besucher-Score.

6.3 Kandidaten im unveränderten Datensatz

Die eingefrorene LSTM-Regel markiert 105 der 9.604 gemessenen Testpunkte. Sie bilden 79 zusammenhängende Forecast-Ereignisse. 76 Punkte erhalten wegen stark imputierter Kontexte oder einer absoluten Überschreitung unter einer Person niedrige Konfidenz. Abbildung 6.3 zeigt den Sensor mit den meisten Forecast-Kandidaten. Die rote Markierung bedeutet eine statistische Abweichung und keine fachlich bestätigte Anomalie.

Der Transformer markiert 91 Testpunkte in 71 Ereignissen; 68 Punkte besitzen niedrige Konfidenz. Beide Modelle weisen zusätzlich 11.636 nicht gemessene Teststunden in 1.460 Datenqualitätsereignissen aus. Alle 30 Sensoren erfüllen im Jahresdatensatz die Mindestmenge für die Modellierung. Forecast- und Datenqualitätsereignisse bleiben getrennt, damit ein fehlender Datenstrom nicht als bestätigte Abweichung der Passantenfrequenz erscheint.

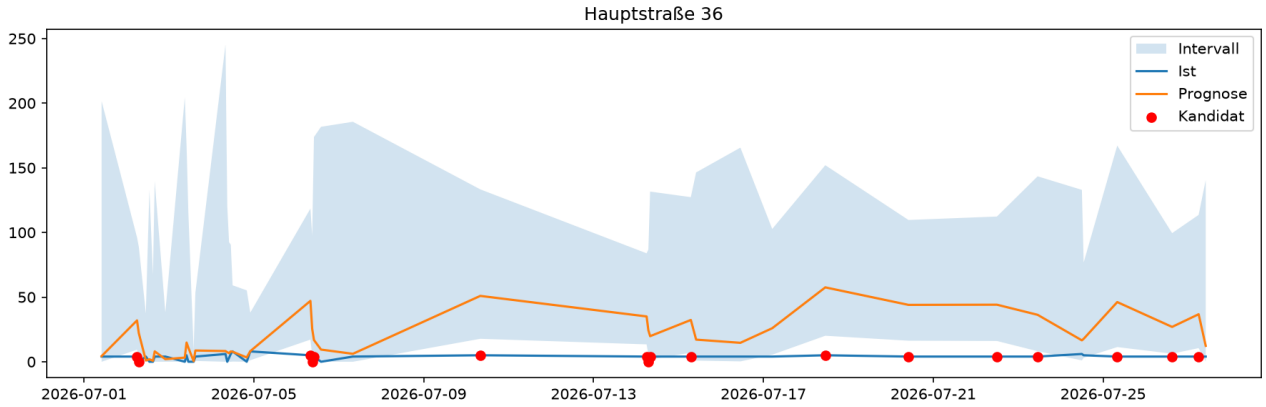


Abbildung 6.3: Messung, Medianprognose, empirisch kalibriertes Intervall und Forecast-Kandidaten eines Beispielsensors. Eigene Darstellung.

Die ereignisbezogenen Prüftabellen enthalten alle 79 LSTM- und 71 Transformer-Ereignisse. Nach dem Jahreslauf stehen sämtliche Einträge wieder auf "offen"; Bewertungen des früheren Zeitraums werden nur bei identischem Sensor, Beginn und Ende übernommen. Weil weder alle gemessenen Testpunkte noch alle Ereignisse vollständig beschriftet sind, werden daraus keine realen Precision-, Recall-, F1- oder AUC-Werte berechnet.

6.4 Ergebnisse der Anomalieinjektion

Tabelle 6.2 fasst für den LSTM-Forecaster die starken Besucher-Injektionen der Stärke 2,0 und den gefilterten sechs Stunden langen Stuck-at-Test über drei Testseeds zusammen. Sämtliche Treffer sind inkrementell gegenüber dem jeweils gepaarten unveränderten Lauf.

Die starken Level-Shifts werden vollständig auf Ereignisebene gefunden. Auch der Drift erreicht einen Ereignis-Recall von 0,972. Stuck-at bleibt mit einem Ereignis-F1 von 0,762 am schwierigsten. Bei Stärke 0,5 beträgt der Ereignis-Recall 0,167 für Spike, 0,185 für Dip, 0,556 für Level-Shift und 0,361 für Drift. Der Detektor erkennt folglich nicht jede definierte schwache Anomalie. Die vollständigen Werte aller Stärken ein-

Tabelle 6.2: Punkt- und Ereignisergebnisse der LSTM-Testinjektionen. R = Recall. Eigene Darstellung.

Typ	Punkt-R	Punkt-F1	Ereignis-R	Ereignis-F1
Spike, 2,0	0,917	0,542	0,917	0,902
Dip, 2,0	0,889	0,667	0,889	0,906
Level-Shift, 2,0	0,979	0,780	1,000	0,911
Drift, 2,0	0,701	0,647	0,972	0,946
Stuck-at, 6 h	0,529	0,627	0,667	0,762

schließlich Precision, Average Precision (AP), IoU und Verzögerung stehen in `injection_benchmark.csv` und `injection_benchmark.json`.

6.5 Vergleich der finalen Modelle

Tabelle 6.3 vergleicht LSTM-Forecaster, Transformer-Forecaster, Autoencoder und robuste saisonale Baseline auf den starken Testinjektionen. Dazu werden die fünf Anomaliearten gleich gewichtet gemittelt. Die künstlichen Testereignisse wurden für kein Verfahren zur Architektur- oder Regelauswahl verwendet.

Tabelle 6.3: Gemeinsamer Vergleich auf starken künstlichen Anomalien. Eigene Darstellung.

Modell	Parameter	ROC-AUC	AP	Punkt-F1	Ereignis-R	Ereignis-F1
LSTM-Forecaster	21.374	0,959	0,463	0,653	0,889	0,885
Transformer	61.062	0,960	0,441	0,603	0,892	0,824
Autoencoder	50.562	0,880	0,241	0,459	0,584	0,547
Saisonal $t-24$	0	0,858	0,118	0,326	0,564	0,478

Der Transformer erreicht die knapp höchste ROC-AUC und den knapp höchsten Ereignis-Recall. Der LSTM besitzt dagegen die höhere AP sowie den höchsten Punkt- und Ereignis-F1. Alle Verfahren werden auf denselben 9.604 gemessenen Testzielen je Lauf bewertet. Die saisonale Baseline kann davon 8.382 mit einer ebenfalls gemessenen $t-24$-Referenz scoren; an den übrigen Punkten enthält sie sich eines Alarms und fehlende Scores werden für die Rankingmetriken als niedrigste Priorität eingeordnet. Im unveränderten Lauf alarmieren LSTM, Transformer, Autoencoder und saisonale Baseline bei 1,093 %, 0,948 %, 1,270 % beziehungsweise 0,448 % der gemessenen Testziele.

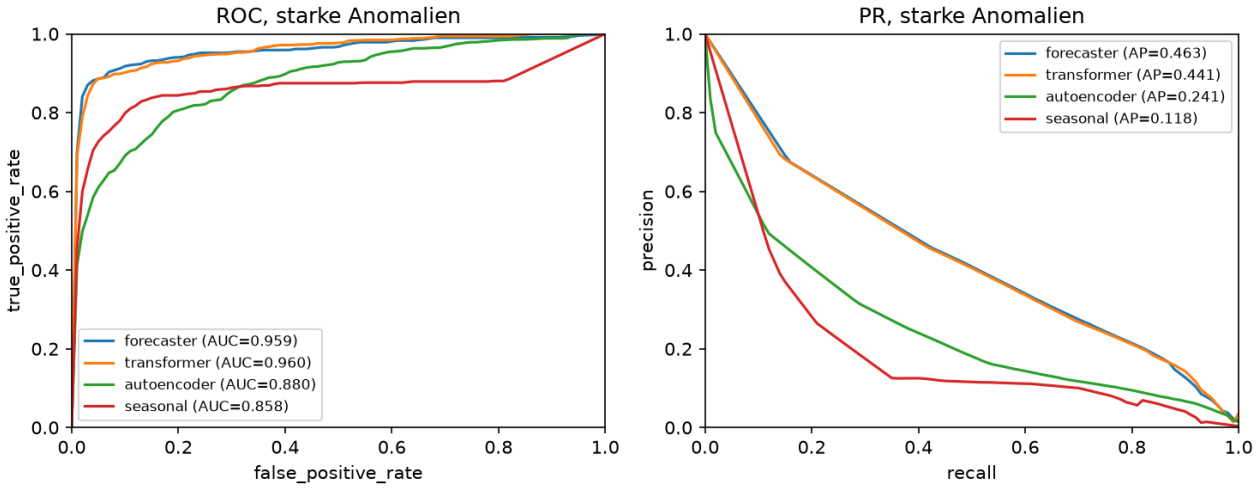


Abbildung 6.4: Mittlere ROC- und PR-Kurven der starken synthetischen Anomalien. Eigene Darstellung.

Im Rauschtest erzeugt der LSTM-Forecaster bei 5 % und 15 % Stärke 13 und 36 zusätzliche Alarme, entsprechend 0,135 % und 0,375 % der gemessenen Testpunkte. Beim Transformer sind es 16 und 25 (0,167 % und 0,260 %), beim Autoencoder 11 und 19 (0,115 % und 0,198 %) und bei der saisonalen Baseline acht und zehn (0,083 % und 0,104 %). Die parameterfreie Baseline ist in diesem Test somit am stabilsten, besitzt aber die schwächeren Erkennungskennzahlen.

6.6 Diskussion und Limitationen

Die Ergebnisse beantworten die Forschungsfrage differenziert. Der Transformer prognostiziert beide Messgrößen genauer und trennt die Rohscores im Mittel minimal besser. Die auf dem Kalibrierungsabschnitt ausgewählte LSTM-Regel liefert auf den Testinjektionen jedoch den höheren Punkt- und Ereignis-F1. Die zusätzliche Attention ist damit keine allgemeine Garantie für bessere binäre Alarme. Die saisonale Methode zeigt den Wert einer einfachen Baseline, und der Autoencoder bleibt trotz größerer Architektur sowie ausgeschöpfter 200 Epochen hinter den Prognosemodellen.

Die synthetische Ground Truth gilt ausschließlich für die definierten Transformationen der Passantenfrequenz. Sie belegt weder den Recall unbekannter realer Anomalien noch die fachliche Richtigkeit der 79 LSTM- beziehungsweise 71 Transformer-Kandidatenereignisse. Der Datensatz umfasst zwar rund 13 Monate, aber nur einen Ort und ungefähr einen Jahreszyklus. Es fehlt eine zeitlich und räumlich unabhängige externe Bestätigung. Zudem wurden 34,46 % der aktiven Sensorstunden imputiert; diese Werte werden nicht als Ziele verwendet, können aber den Kontext beeinflussen. Zeitliche Abhängigkeit verhindert eine klassische verteilungsfreie Conformal-Garantie. Schließlich wurde der feste einmonatige Testzeitraum im Verlauf der retrospektiven Entwicklung bereits betrachtet. Neue Zufallsstartwerte trennen die Testinjektionen von der Kalibrierung, ersetzen aber keine konfirmatorische Studie mit realen Labels und mehreren Rolling-Origin-Ursprüngen.

Kapitel 7

Zusammenfassung und Ausblick

Die aufbereiteten Sensordaten wurden mit vier getrennten Verfahren untersucht: Ein LSTM und ein kompakter Transformer bewerten Abweichungen von ihrer Prognose, ein Autoencoder bewertet Rekonstruktionsfehler und eine robuste parameterfreie Baseline die Abweichung vom gemessenen Wert 24 Stunden zuvor. Die drei neuronalen Modelle lernen zusätzlich die Verweildauer; sämtliche Alarme und künstlichen Testanomalien beziehen sich aber ausschließlich auf die Passantenfrequenz. Die Verfahren verwenden dieselben Zeitabschnitte und Entscheidungskriterien, ohne ihre Anomaliewerte miteinander zu vermischen.

Der Transformer prognostiziert beide Zielgrößen am genauesten und erreicht auf starken synthetischen Anomalien die knapp höchste mittlere ROC-AUC. Der LSTM erreicht mit der kalibrierten Alarmregel dagegen den höchsten Punkt- und Ereignis-F1. Die saisonale Baseline reagiert am robustesten auf zusätzliches Zählwertrauschen, bleibt bei der Erkennung der injizierten Ereignisse aber hinter den Prognosemodellen. Trotz einer größeren gewählten Architektur und 200 ausgeschöpften Epochen erreicht auch der Autoencoder deren Gesamtkennzahlen nicht. Zusätzliche Attention verbessert damit Prognose und Ranking der Rohscores, führt aber nicht automatisch zur besten binären Alarmregel.

Als nächste Schritte sind vor allem die vollständige manuelle Prüfung der 79 LSTM- und 71 Transformer-Kandidatenereignisse, die Klärung der geplanten Sensorlaufzeiten und eine Bestätigung auf einem zeitlich sowie möglichst räumlich unabhängigen Datensatz sinnvoll. Weitere Entwicklungsläufe können mit einem kleineren Autoencoder-Budget und einem Ein-Seed-Smoke-Benchmark beschleunigt werden; die hier berichtete Referenzauswertung sollte für den abschließenden Vergleich vollständig bleiben. Erst reale Ereignislabels erlauben belastbare Aussagen zur Erkennung unbekannter fachlicher Anomalien.

Literatur

[1] Rina Foygel Barber u. a. "Conformal Prediction Beyond Exchangeability". In: *The Annals of Statistics* 51.2 (2023), S. 816–845. DOI: 10.1214/23-AOS2276.

[2] Yuchen Fang u. a. "Temporal-Frequency Masked Autoencoders for Time Series Anomaly Detection". In: *2024 IEEE 40th International Conference on Data Engineering*. 2024, S. 1228–1241. DOI: 10.1109/ICDE60146.2024.00099.

[3] Jan Gasthaus u. a. "Probabilistic Forecasting with Spline Quantile Function RNNs". In: *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*. Bd. 89. Proceedings of Machine Learning Research. 2019, S. 1901–1910. URL: https://proceedings.mlr.press/v89/gasthaus19a.html.

[4] Sepp Hochreiter und Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), S. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[5] Kyle Hundman u. a. "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2018, S. 387–395. DOI: 10.1145/3219819.3219845.

[6] Rob J. Hyndman und Anne B. Koehler. "Another Look at Measures of Forecast Accuracy". In: *International Journal of Forecasting* 22.4 (2006), S. 679–688. DOI: 10.1016/j.ijforecast.2006.03.001.

[7] Siwon Kim u. a. "Towards a Rigorous Evaluation of Time-Series Anomaly Detection". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 36.7 (2022), S. 7194–7201. DOI: 10.1609/aaai.v36i7.20680.

[8] Roger Koenker und Jr. Bassett Gilbert. "Regression Quantiles". In: *Econometrica* 46.1 (1978), S. 33–50. DOI: 10.2307/1913643.

[9] Bryan Lim u. a. "Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting". In: *International Journal of Forecasting* 37.4 (2021), S. 1748–1764. DOI: 10.1016/j.ijforecast.2021.03.012.

[10] Qinghua Liu und John Paparrizos. "The Elephant in the Room: Towards a Reliable Time-Series Anomaly Detection Benchmark". In: *Advances in Neural Information Processing Systems*. Bd. 37. 2024. DOI: 10.52202/079017-3437.

[11] Pankaj Malhotra u. a. "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection". In: *ICML 2016 Anomaly Detection Workshop*. 2016. DOI: 10.48550/arXiv.1607.00148.

[12] Yuqi Nie u. a. "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=Jbdc0vTOcol.

[13] Yaniv Romano, Evan Patterson und Emmanuel J. Candès. "Conformalized Quantile Regression". In: *Advances in Neural Information Processing Systems*. Bd. 32. 2019, S. 3543–3553. URL: https://proceedings.neurips.cc/paper_files/paper/2019/hash/5103c3584b063c431bd1268e9b5e76fb-Abstract.html.

[14] David Salinas u. a. "DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks". In: *International Journal of Forecasting* 36.3 (2020), S. 1181–1191. DOI: 10.1016/j.ijforecast.2019.07.001.

[15] M. Saquib Sarfraz u. a. "Position: Quo Vadis, Unsupervised Time Series Anomaly Detection?" In: *Proceedings of the 41st International Conference on Machine Learning*. Bd. 235. Proceedings of Machine Learning Research. 2024, S. 43461–43476. URL: https://proceedings.mlr.press/v235/sarfraz24a.html.

[16] Sebastian Schmidl, Phillip Wenig und Thorsten Papenbrock. "Anomaly Detection in Time Series: A Comprehensive Evaluation". In: *Proceedings of the VLDB Endowment* 15.9 (2022), S. 1779–1797. DOI: 10.14778/3538598.3538602.

[17] Muhammad Awais Shafique. "Imputing Missing Data in Hourly Traffic Counts". In: *Sensors* 22.24 (2022), S. 9876. DOI: 10.3390/s22249876.

[18] Nitish Srivastava u. a. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.56 (2014), S. 1929–1958. URL: https://www.jmlr.org/papers/v15/srivastava14a.html.

[19] Tuo Sun u. a. "Traffic Missing Data Imputation: A Selective Overview of Temporal Theories and Algorithms". In: *Mathematics* 10.14 (2022), S. 2544. DOI: 10.3390/math10142544.

[20] Nesime Tatbul u. a. "Precision and Recall for Time Series". In: *Advances in Neural Information Processing Systems*. Bd. 31. 2018, S. 1920–1930. URL: https://proceedings.neurips.cc/paper_files/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html.

[21] Ashish Vaswani u. a. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Bd. 30. Curran Associates, Inc., 2017, S. 5998–6008. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[22] Pascal Vincent u. a. "Extracting and Composing Robust Features with Denoising Autoencoders". In: *Proceedings of the 25th International Conference on Machine Learning*. 2008, S. 1096–1103. DOI: 10.1145/1390156.1390294.

[23] Ruibin Xiong u. a. "On Layer Normalization in the Transformer Architecture". In: *Proceedings of the 37th International Conference on Machine Learning*. Bd. 119. Proceedings of Machine Learning Research. PMLR, 2020, S. 10524–10533. URL: https://proceedings.mlr.press/v119/xiong20b.html.

[24] Ailing Zeng u. a. "Are Transformers Effective for Time Series Forecasting?" In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.9 (2023), S. 11121–11128. DOI: 10.1609/aaai.v37i9.26317.

[25] Yimei Zhang u. a. "A Comprehensive Survey on Traffic Missing Data Imputation". In: *IEEE Transactions on Intelligent Transportation Systems* 25.12 (2024), S. 19252–19275. DOI: 10.1109/TITS.2024.3478816.